

An Area-Efficient Pipelined Signed Radix-4 Modified Russian Peasant Multiplier for Biomedical FIR Filtering

Aditya Garg, Ashu Anand, Gargi Singh, Aditya Mohan and Jyoti Yadav

Dept. of Instrumentation and Control Engineering

Netaji Subhas University of Technology (NSUT)

Delhi, India

Email: {aditya.garg.ug23, ashu.anand.ug23, gargi.singh.ug23, aditya.mohan.ug23, jyoti.yadav}@nsut.ac.in

Abstract—Hardware multipliers dominate the area and power budget of the digital signal-processing chains used in wearable and implantable biomedical devices, where silicon area and battery life are tightly constrained. The Modified Russian Peasant Multiplier (MRPM) has recently been shown to be an attractive low-area primitive for such systems. This work extends that architecture in four directions. First, the iterative unsigned MRPM is replaced by a *signed radix-4 modified-Booth* formulation that reduces an 8-bit multiplication to four partial products evaluated in parallel, rather than eight sequential shift-and-add iterations, while natively supporting two's-complement operands. Second, the linear-phase symmetry of the filter coefficients is exploited to *fold* an 8-tap FIR onto four multipliers instead of eight, reducing LUT4 utilisation of the complete filter from 345 to 222 - a saving of 35.7% - with bit-identical output. Third, the folded datapath is pipelined into four stages, raising the maximum clock frequency by $1.55\times$ (42.96 MHz to 66.47 MHz). Fourth, four parallel-prefix adder topologies are synthesised on a common fabric; we find that at 8-bit datapath width the choice of prefix adder is close to immaterial (222–238 LUT4, 41.8–45.0 MHz), which suggests that effort is better directed at the partial-product count and the filter structure than at the adder. The design is exact rather than approximate: the multiplier is verified exhaustively over all 65,536 signed operand pairs with zero mismatches, and every filter configuration matches an independent golden model bit-for-bit. Synthesis and place-and-route for a low-cost Gowin GW1NR-9C FPGA use LUT logic exclusively, with dedicated DSP inference disabled and zero DSP blocks confirmed in every run, making the architecture suitable for area-constrained ECG and EEG front ends.

Index Terms—Multiplier, Russian Peasant algorithm, Booth recoding, parallel-prefix adder, parallel-prefix adder, FIR filter, FPGA, biomedical signal processing, VLSI.

I. INTRODUCTION

The design of efficient hardware multipliers remains central to digital signal processing, because the multiplier is typically the most area- and power-intensive element of an arithmetic datapath [2], [3]. This constraint is felt most acutely in wearable health monitors and implantable biomedical devices, where every square millimetre of silicon and every microampere of current directly limits device lifetime and viability [4]. Such systems must simultaneously satisfy three requirements that ordinarily trade off against one another: computational accuracy, resource efficiency, and low power.

Classical multiplier architectures - array, Booth, Wallace-tree and Vedic designs [5]–[8] — have been studied extensively, but their resource utilisation remains high for resource-constrained deployment. A second family of designs achieves area and power reduction by *approximating* the product [9], [10]. Approximate multipliers are effective for error-tolerant workloads such as image processing, but they are unsuitable for diagnostic biomedical signal chains, where a small numerical deviation can alter clinical interpretation. An exact, low-area multiplier is therefore required.

The Russian Peasant Multiplication (RPM) algorithm offers one route to such a primitive. RPM computes a product by repeatedly halving one operand and doubling the other, accumulating the doubled operand whenever the halved operand is odd. Because it reuses a small set of shift and add resources rather than generating and storing a full partial-product array, RPM is inherently compact. Recent work by TN, Deepika Sree, and S. Aditya [1] proposed a Modified Russian Peasant Multiplier (MRPM) that couples this algorithm with a Kogge-Stone parallel-prefix adder, and demonstrated its suitability for biomedical applications on an FPGA platform. That work is the direct predecessor of the present paper.

Building on [1], we observe five opportunities for further optimisation that the base architecture leaves open:

- 1) **Unsigned operands only:** Biomedical signal processing routinely involves differential and mean-removed signals, which are inherently signed. An unsigned multiplier requires external sign handling, adding logic outside the multiplier boundary.
- 2) **Iteration-bound throughput:** RPM consumes approximately one multiplier bit per iteration, requiring on the order of eight iterations for an 8-bit operand. Accelerating the *adder* inside a sequential loop reduces only the per-iteration cost; the iteration count itself remains the dominant term in end-to-end latency.
- 3) **No radix recoding:** The partial-product count is not reduced by radix-4 (or higher) recoding, although this is a standard and well-understood lever.
- 4) **No structural FIR optimisation:** When the multiplier is deployed in an FIR filter, the linear-phase symmetry

of the coefficient set is not exploited to reduce the multiplier count.

- 5) **Adder choice not swept:** A Kogge-Stone adder is adopted without a comparative evaluation against other parallel-prefix topologies on the same fabric. Since the depth/cell-count/fan-out trade-off among prefix networks is strongly width-dependent, it is not self-evident that the minimum-depth topology is the right choice at 8-bit width.

These are opportunities for extension rather than deficiencies: the base architecture establishes MRPM as a viable low-area primitive, and this work asks how much further the same idea can be pushed.

Contributions. This paper makes the following contributions:

- 1) A **signed radix-4 modified-Booth MRPM**, reducing an 8-bit multiplication to four partial products and supporting two's-complement operands natively. Correctness is established *exhaustively* over all 65,536 signed operand pairs.
- 2) A **comparative synthesis sweep** of four parallel-prefix adder topologies (Kogge-Stone, Brent-Kung, Sklansky and a Han-Carlson-scheduled network) within an otherwise identical filter on a common fabric, establishing that at 8-bit width the prefix topology has only a marginal effect on area, delay and power.
- 3) A **symmetric-folded 8-tap linear-phase FIR** that requires only four multipliers instead of eight, producing *bit-identical* output.
- 4) A **four-stage pipelined** realisation of the folded datapath, shortening the critical path while leaving the functional output unchanged.
- 5) A **LUT-only implementation** targeting a low-cost Gowin GW1NR-9C FPGA, with dedicated DSP inference explicitly disabled and zero DSP blocks confirmed post-place-and-route, so that the reported area reflects genuine logic-fabric cost.

The remainder of this paper is organised as follows. Section II reviews related multiplier and adder architectures and positions this work with respect to [1]. Section III develops the proposed architecture. Section IV describes the number formats, bit-growth analysis and implementation flow. Section V presents the functional verification methodology and results. Section VI reports synthesis results. Section VII concludes.

II. RELATED WORK

A. Multiplier architectures

Booth recoding [6] and Wallace-tree reduction [11] remain the standard techniques for reducing, respectively, the number of partial products and the depth of their summation. Vedic multipliers [7], [8] exploit parallel sub-product formation and are frequently reported for FPGA targets; they favour speed at the cost of a larger hardware footprint, which limits their attractiveness in area-constrained biomedical deployment. Phuse and Tasgaonkar [5] provide a comparative FPGA evaluation of array, Booth and Vedic multipliers within a MAC unit, and

Badawi *et al.* [12] evaluate a fixed-width modified Baugh-Wooley multiplier.

A distinct line of work trades accuracy for area. Khurshid [9] proposes a resource-optimal approximate multiplier for error-resilient applications, and multi-level approximate multipliers using modified full adders have been reported with substantial power, area and gate-count reductions [10]. These designs are well matched to image processing, but the error tolerance they assume is not acceptable in diagnostic biomedical processing, where the present work requires exactness.

Paz and Garrido [13] implement complex multipliers efficiently on FPGAs by mapping them onto dedicated DSP slices. That approach is effective when DSP resources are plentiful, but it is orthogonal to the goal pursued here: on small, low-cost devices the DSP blocks are few and are typically reserved for other datapath functions, so a multiplier realised purely in general-purpose LUT logic is the more useful primitive. This motivates the explicit DSP-disabled synthesis flow adopted in Section IV.

B. Russian Peasant multipliers

The RPM algorithm has been revisited several times as a low-area alternative to array and tree multipliers. Sharma and Singh [2] apply a Peasant-multiplication technique to signal processing. Kumar and Rabi [14] implement a modified RPM using an MSQRTCSLA-based adder within an FIR filter. Gunasekaran and Manikandan [15] construct a high-speed reconfigurable FIR filter around an RPM, replacing the Wallace adder with a carry-select/Sklansky combination and reporting reductions in latency, power and area; that work is the closest prior art to the FIR deployment considered here. Rao *et al.* [16] pair a modified Russian Peasant multiplier with a Han-Carlson adder to shorten partial-product addition, reporting a substantial speed improvement relative to RPM realisations based on ripple-carry, carry-save and compressor-based adders. Reference [16] is the principal precedent for the adder choice made in this paper.

TN, Deepika Sree, and S. Aditya [1] combine a modified RPM with a Kogge-Stone adder and target biomedical workloads on an FPGA, reporting substantially reduced device utilisation relative to conventional multipliers. The present work adopts the same overall premise — that RPM-derived multipliers are well suited to area- and power-constrained biomedical hardware — and extends it along the five axes enumerated in Section I.

C. Parallel-prefix adders

Parallel-prefix adders differ in how they trade logic depth against prefix-cell count and fan-out. Kogge-Stone attains minimal depth ($\log_2 N$) at the cost of the largest cell count and the heaviest wiring; Brent-Kung minimises cell count at roughly twice the depth; Sklansky attains minimal depth with fewer cells but with fan-out that grows across levels; Han-Carlson hybridises the Kogge-Stone and Brent-Kung schedules, achieving depth $\approx \log_2 N + 1$ with substantially fewer cells and bounded fan-out. Panchalaiah and Kumar [17] report

a high-speed, energy-efficient Kogge-Stone design across 8- to 64-bit widths, and Raju and Sa [18] analyse multipliers built around a Kogge-Stone adder. Anjana *et al.* [19] combine a Vedic multiplier with a Kogge-Stone adder.

The width-dependence of this trade-off is central to the present work. At the narrow widths characteristic of an 8-bit biomedical datapath, the depth advantage of Kogge-Stone over Han-Carlson amounts to a single prefix level, whereas its cell-count and routing overheads do not diminish. This motivates both the adoption of Han-Carlson and the four-way comparative sweep reported in Section VI.

III. PROPOSED ARCHITECTURE

A. Design context

The target application is the front-end low-pass filter of an electrocardiogram (ECG) acquisition chain. We assume a sampling rate of $f_s = 250$ Hz, typical of wearable ECG acquisition, and a low-pass cut-off of $f_c = 40$ Hz, which suppresses electromyographic and high-frequency noise while retaining the diagnostic band. The filter is an 8-tap linear-phase FIR designed by the windowed-sinc method with a Hamming window. Linear phase is not merely convenient: a non-linear phase response distorts the morphology of the QRS complex, which is the feature on which most downstream ECG analysis depends. The resulting coefficient set, quantised to 8-bit signed integers with a scale factor of 2^7 , is

$$h = [0, 3, 20, 42, 42, 20, 3, 0], \quad (1)$$

which satisfies the symmetry condition $h[i] = h[N - 1 - i]$ with $N = 8$, and has a DC gain of $130/128 \approx 1.016$. The symmetry in (1) is exploited structurally in Section III-D.

B. Signed radix-4 modified-Booth MRPM

The base architecture [1] evaluates one multiplier bit per iteration, requiring approximately N iterations for an N -bit operand. We instead recode the multiplier in radix-4, consuming two bits per step and thereby producing $N/2 = 4$ partial products for $N = 8$. Radix-4 modified-Booth recoding is adopted rather than an unsigned radix-4 scheme with a pre-computed $3B$ term, because Booth recoding restricts the digit set to $\{-2, -1, 0, +1, +2\}$ — eliminating the awkward $3B$ multiple entirely — and handles two's-complement operands natively, which simultaneously closes gaps (1) and (3) of Section I.

For each step $k \in \{0, 1, 2, 3\}$ the recoder examines the bit triplet $(b_{2k+1}, b_{2k}, b_{2k-1})$, where the phantom bit $b_{-1} \equiv 0$. The control signals are

$$\text{one}_k = b_{2k} \oplus b_{2k-1}, \quad (2)$$

$$\text{two}_k = (b_{2k+1} \cdot \overline{b_{2k}} \cdot \overline{b_{2k-1}}) + (\overline{b_{2k+1}} \cdot b_{2k} \cdot b_{2k-1}), \quad (3)$$

$$\text{neg}_k = b_{2k+1}. \quad (4)$$

The recoding rules (2), (3) and (4) produce a signed digit in $\{-2, -1, 0, +1, +2\}$ at each step. The signals one_k and two_k

Algorithm 1 Signed radix-4 MRPM with Han-Carlson accumulation

Require: Signed N -bit multiplicand A , signed N -bit multiplier B

Ensure: Signed $2N$ -bit product P

```

1:  $A_{\text{ext}} \leftarrow \text{signext}(A, 2N)$ 
2:  $b_{-1} \leftarrow 0$  {phantom bit}
3: for  $k = 0$  to  $N/2 - 1$  do
4:    $\text{one}_k \leftarrow b_{2k} \oplus b_{2k-1}$ 
5:    $\text{two}_k \leftarrow (b_{2k+1} \overline{b_{2k}} \overline{b_{2k-1}}) + (\overline{b_{2k+1}} b_{2k} b_{2k-1})$ 
6:    $\text{neg}_k \leftarrow b_{2k+1}$ 
7:    $M_k \leftarrow \text{one}_k \cdot A_{\text{ext}} \vee \text{two}_k \cdot (A_{\text{ext}} \ll 1)$ 
8:    $\text{PP}_k \leftarrow (\text{neg}_k ? (\overline{M_k} + 1) : M_k) \ll 2k$ 
9: end for
10:  $S_0 \leftarrow \text{HanCarlson}(\text{PP}_0, \text{PP}_1)$ 
11:  $S_1 \leftarrow \text{HanCarlson}(\text{PP}_2, \text{PP}_3)$ 
12:  $P \leftarrow \text{HanCarlson}(S_0, S_1)$ 
13: return  $P$ 
```

are mutually exclusive by construction, so the partial-product magnitude is formed by selection rather than addition:

$$M_k = \text{one}_k \cdot A \vee \text{two}_k \cdot (A \ll 1), \quad (5)$$

where A is the sign-extended multiplicand. The signed partial product is then obtained by conditional two's-complement negation and alignment:

$$\text{PP}_k = (\text{neg}_k ? (\overline{M_k} + 1) : M_k) \ll 2k. \quad (6)$$

The four aligned partial products are summed by a three-node Han-Carlson adder tree of depth two, as set out in Algorithm 1. Because the doubling in (5) and the alignment in (6) are fixed shifts, they are realised as hard-wired routing rather than as barrel shifters, which is the principal structural saving relative to an iterative RPM datapath.

C. Parallel-prefix addition

All additions — both the partial-product accumulation of Algorithm 1 and the filter summation of Section III-D — are performed by a parallel-prefix adder. Pre-processing forms the bitwise generate and propagate terms of (7),

$$g_i = a_i \cdot b_i, \quad p_i = a_i \oplus b_i. \quad (7)$$

A logarithmic-depth prefix network then combines these terms; the carry into bit $i + 1$ and the sum bit follow as (8),

$$c_{i+1} = G_i + (P_i \cdot c_{\text{in}}), \quad s_i = p_i \oplus c_i, \quad (8)$$

in which (G_i, P_i) denote the group generate and propagate signals at the output of the prefix network.

Prefix topologies differ in how they trade depth against cell count and fan-out. Kogge-Stone attains minimal depth ($\log_2 W$ for width W) at the cost of the largest cell count and the heaviest routing. Brent-Kung minimises cell count at roughly twice the depth. Sklansky attains minimal depth with fewer

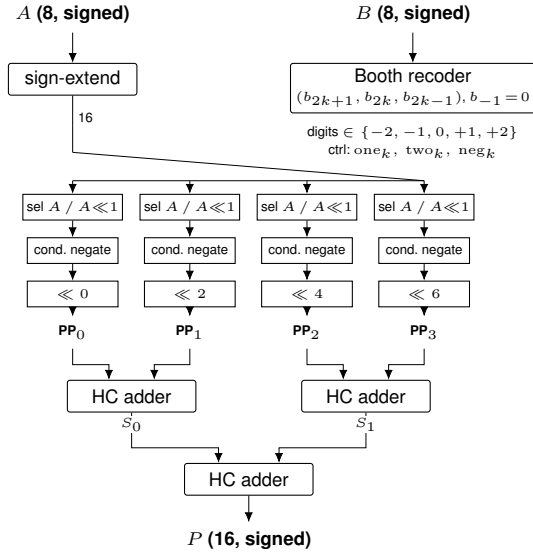


Fig. 1. Datapath of the proposed signed radix-4 MRPM. The multiplier operand is recoded two bits at a time into the digit set $\{-2, -1, 0, +1, +2\}$, yielding four partial products that are aligned by hard-wired shifts and summed by a two-level parallel-prefix adder tree.

cells, at the cost of fan-out that grows across levels. Han-Carlson hybridises the Kogge-Stone and Brent-Kung schedules, targeting depth $\approx \log_2 W + 1$ with substantially fewer cells.

Rather than assume a topology, we treat the choice as an empirical question and synthesise all four within an otherwise identical filter (Section VI). The four adders share a common port list and are selected at compile time, so that any measured difference is attributable to the prefix network alone.

Disclosure on the realised networks. The adders are described behaviourally, and the structure committed to the fabric is determined by the synthesiser rather than hand-instantiated at the prefix-cell level. One consequence must be stated plainly: the network we describe under a Han-Carlson schedule and the Kogge-Stone network synthesise to the *same* netlist on this fabric, and therefore yield identical area, timing and power. We accordingly report them as a single design point in Section VI rather than as two, and we make no claim to have realised a distinct Han-Carlson prefix structure. The comparison that follows is therefore between three distinct synthesised prefix structures — Kogge-Stone (equivalently, the Han-Carlson-scheduled description), Brent-Kung and Sklansky — all derived from functionally equivalent descriptions and all verified to produce identical arithmetic results (Section V).

D. Symmetric-folded FIR filter

For a linear-phase FIR the coefficient set is symmetric, $h[i] = h[N - 1 - i]$, as is the case for (1). The direct-form output

$$y[n] = \sum_{i=0}^{N-1} h[i] x[n-i] \quad (9)$$

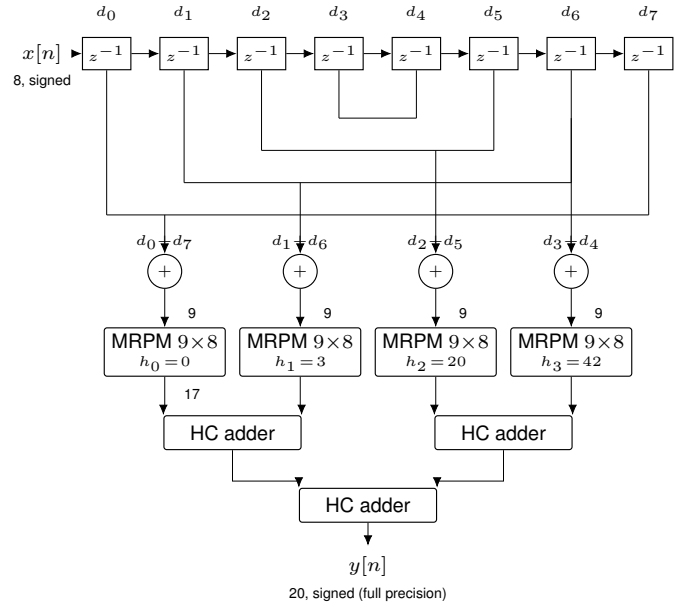


Fig. 2. Symmetric-folded 8-tap linear-phase FIR. Mirror-image taps (d_i with d_{7-i}) are pre-added before multiplication, halving the multiplier count from eight to four with bit-identical output. Each pre-adder widens the operand to nine bits, so the surviving multipliers are 9×8 ; this is why the measured LUT4 saving is 35.7% rather than 50%.

can therefore be regrouped by pairing each tap with its mirror image:

$$y[n] = \sum_{i=0}^{N/2-1} h[i] (x[n-i] + x[n-(N-1-i)]). \quad (10)$$

Equation (10) is an algebraic identity, not an approximation: it replaces N multiplications by $N/2$ multiplications and $N/2$ pre-additions, and its output is *bit-identical* to (9). For $N = 8$ the multiplier count falls from eight to four. Since the multiplier is by a wide margin the dominant consumer of logic resources in the filter, and a pre-adder is far cheaper than a multiplier, this is the principal area lever of the design.

Each pre-add sums two signed 8-bit samples and therefore requires one guard bit, yielding a 9-bit signed operand; the multipliers are correspondingly 9×8 . The four products are summed by a three-node Han-Carlson tree to a full-precision 20-bit signed output (Section IV). The resulting structure is shown in Fig. 2.

E. Pipelining

In its unpipelined form the folded filter presents a single combinational path that traverses a complete multiplier and the full summation tree, so the achievable clock frequency is bounded by the sum of both. The datapath is therefore partitioned into four pipeline stages, with register boundaries placed as follows:

- **S1** — the input delay line (already registered);
- **S2** — within each multiplier, between Booth partial-product generation (6) and the accumulation tree;

TABLE I
FIXED-POINT WIDTHS AND BIT-GROWTH

Quantity	Width	Rationale
Input x , coefficient h	8-bit signed	range $-128 \dots 127$
Pre-add $x[n-i] + x[n-7+i]$	9-bit signed	one guard bit
Product (9×8)	17-bit signed	—
Output y	20-bit signed	(11); full precision

- **S3** — after the multipliers, at the inputs of the first level of the filter summation tree;
- **S4** — after the first summation level, feeding the registered output.

The critical path is thereby reduced to approximately one adder level. Latency rises from one cycle to four, while throughput remains one sample per clock cycle in steady state; the functional output is unchanged, which is confirmed in Section V.

IV. IMPLEMENTATION

A. Number formats and bit-growth

The filter is realised in fixed-point arithmetic with the widths of Table I. Input samples and coefficients are 8-bit signed integers; the coefficients of (1) carry an implicit scale factor of 2^7 , so the datapath output is the true filter response scaled by 128, and a deployed system applies a final arithmetic right-shift of seven places.

Bit-growth is bounded as follows. Each mirror-pair pre-add of two signed 8-bit samples requires one guard bit, giving a 9-bit operand. Each 9×8 product is 17 bits. The worst-case magnitude of the accumulated sum is bounded by

$$|y|_{\max} \leq N \cdot |x|_{\max} \cdot |h|_{\max} = 8 \cdot 127 \cdot 128 = 130,048, \quad (11)$$

which requires 18 bits in two's-complement representation. A 20-bit output is adopted, providing headroom ($2^{19} = 524,288$) and retaining full precision: no rounding, truncation or saturation is applied anywhere in the datapath, so the filter is numerically exact with respect to its integer specification.

B. Design and verification flow

The architecture is described in synthesisable Verilog. Functional simulation and regression are performed with Icarus Verilog, with waveform inspection in GTKWave. An independent golden model, written in Python/NumPy, implements the radix-4 Booth recoding and the fixed-point filter arithmetic; it is itself cross-checked against direct integer convolution, so that the reference used to validate the RTL does not share an implementation lineage with it. The complete regression suite is driven by a single build target.

C. Target device and DSP-disabled implementation flow

The design targets the Gowin GW1NR-9C FPGA (8640 LUT4, 6480 flip-flops), the device carried by the Sipeed Tang Nano 9K board [20] and a substantially lower-cost, lower-capacity part than the Xilinx Artix-7 used in [1]. Results

are reported after synthesis and place-and-route; the design was not downloaded to the device, and no claim of on-chip execution is made.

One methodological point is essential to the validity of the area claim. The GW1NR-9 fabric contains dedicated hardware multiplier (DSP) resources, and a synthesiser will by default infer them from a multiplication operator. Were this permitted, the reported LUT utilisation would not reflect the cost of the proposed multiplier at all, and the central claim of the paper — that the architecture is an area-efficient *LUT-based* multiplier — would be vacuous. Synthesis is therefore constrained to logic-only mapping (`syn_multstyle = "logic"`), and the utilisation report is required to show zero DSP blocks in use. This constraint is applied to every configuration reported in Section VI, including those used for the comparative adder sweep, so that all rows of that comparison are measured on the same footing.

V. FUNCTIONAL VERIFICATION

Because the proposed design is an *exact* multiplier — in deliberate contrast to the approximate architectures of [9], [10] — its correctness must be established without residual error tolerance. Verification proceeds at three levels.

A. Exhaustive multiplier verification

The signed radix-4 MRPM of Section III-B is verified *exhaustively*. For an 8-bit signed multiplier the operand space is finite and small: all $256 \times 256 = 65,536$ ordered pairs (a, b) with $a, b \in [-128, 127]$ are applied to the RTL and the 16-bit product is compared against the reference product $a \cdot b$. The design produces **zero mismatches** across the entire operand space. Exhaustive coverage is a stronger guarantee than the best-case/worst-case/random-vector methodology usually reported for multipliers of this width, since it admits no untested corner.

B. Filter verification against an independent golden model

The symmetric-folded 8-tap filter of Section III-D is verified against the Python golden model over a pseudo-random signed input sequence. The RTL output matches the golden reference with **zero mismatches**, confirming both the correctness of the 9×8 multipliers and the exactness of the folding identity (10): the folded four-multiplier filter and the direct eight-multiplier form produce bit-identical output, so the halving of the multiplier count carries no numerical cost whatsoever.

C. Pipeline and adder-variant equivalence

Two further equivalence checks are performed. First, the four-stage pipelined filter of Section III-E is compared against the same golden reference with the output alignment adjusted for the increased latency; it likewise produces **zero mismatches**, confirming that pipelining alters timing alone and not function. Second, the filter is rebuilt with each prefix-adder description — Han-Carlson-scheduled, Kogge-Stone, Brent-Kung and Sklansky — and each configuration is re-verified; all pass with **zero mismatches**. The adder sweep of Section VI

TABLE II
FUNCTIONAL VERIFICATION RESULTS

Design under test	Method	Mismatches
Signed radix-4 MRPM (8×8)	exhaustive, 65,536 pairs	0
Folded 8-tap FIR	golden model	0
Pipelined folded FIR	golden model, latency-4	0
FIR, Han-Carlson-scheduled adder	golden model	0
FIR, Kogge-Stone adder	golden model	0
FIR, Brent-Kung adder	golden model	0
FIR, Sklansky adder	golden model	0

TABLE III
STRUCTURAL COMPARISON WITH THE BASE ARCHITECTURE

Design	Mult. in FIR	PPs / iter.	Signed
Base MRPM [1]	8	~8 iterations	No
This work, direct	8	4 partial products	Yes
This work, folded	4	4 partial products	Yes

therefore compares *functionally equivalent* implementations, so any difference observed in area, delay or power is attributable to the prefix topology alone.

D. Summary

Table II summarises the verification outcomes. Every functional claim made in this paper is supported by simulation evidence; the structural claims — four partial products rather than eight iterations, four multipliers rather than eight, signed operand support — follow directly from the architecture and are independent of synthesis.

E. Structural comparison

Independently of synthesis, the proposed architecture differs structurally from [1] as summarised in Table III. These figures follow from the algorithm and the filter topology and require no measurement.

VI. SYNTHESIS RESULTS AND DISCUSSION

All configurations were synthesised and placed-and-routed for the Gowin GW1NR-9C, the device carried by the Sipeed Tang Nano 9K board. In every run, DSP inference was disabled and the utilisation report was checked to confirm that **zero DSP blocks** were used; the reported LUT4 figures therefore reflect the true cost of realising the arithmetic in general-purpose logic. All figures below are read directly from Gowin post-place-and-route utilisation, timing and power reports. We report implementation results at the place-and-route level; on-chip execution of the filter is outside the scope of this study and is noted as future work.

A. Effect of symmetric folding

Table IV compares the direct 8-multiplier filter against the symmetric-folded 4-multiplier filter. Folding reduces LUT4 utilisation from 345 to 222, a saving of **35.7%**, while flip-flop count, power and $F_{\max}$ are essentially unchanged. The output is bit-identical (Section V), so this saving is obtained at no numerical cost whatsoever.

TABLE IV
RESOURCE UTILISATION AND TIMING (TANG NANO 9K, GW1NR-9C, DSP DISABLED)

Configuration	LUT4	FF	DSP	$F_{\max}$ (MHz)	Power (mW)
Direct FIR (8 mult.)	345	44	0	42.858	2.582
Folded FIR (4 mult.)	222	44	0	42.956	2.595
Folded + pipelined	324	159	0	66.471	2.639

It is worth being precise about why the saving is 35.7% and not the 50% that a naive reading of “eight multipliers become four” would suggest. Two effects erode the ideal figure. First, folding widens the multiplicand: the pre-add of two signed 8-bit samples produces a 9-bit operand, so the four surviving multipliers are 9×8 rather than 8×8 , and each is individually more expensive than the units it replaces. Second, the four mirror-pair pre-adders are new logic that did not exist in the direct form. The realised transformation is thus given by (12),

$$8 \cdot M_{8 \times 8} \longrightarrow 4 \cdot M_{9 \times 8} + 4 \cdot A_9, \quad (12)$$

rather than a clean halving. That 35.7% survives both effects confirms that the multiplier remains the dominant resource and that trading multipliers for pre-adders is strongly favourable.

B. Effect of pipelining

The four-stage pipeline raises $F_{\max}$ from 42.96 MHz to 66.47 MHz, a speed-up of **1.55×**, at a cost of 115 additional flip-flops (44 to 159) and 102 additional LUT4 (222 to 324), with latency rising from one cycle to four and throughput unchanged at one sample per cycle. Dynamic power rises only marginally, from 2.595 mW to 2.639 mW. The pipelined design remains smaller than the unpipelined *direct* filter (324 versus 345 LUT4) while running 55% faster, so folding and pipelining together dominate the direct form on both axes simultaneously.

C. Prefix-adder comparison

Table V reports the folded filter rebuilt around each prefix topology. As disclosed in Section III-C, the Han-Carlson-scheduled description and the Kogge-Stone network synthesise to the same netlist and are therefore reported as a single point.

The result is, in our view, the most useful finding of the sweep, and it is a largely negative one: *at 8-bit datapath width the choice of prefix adder scarcely matters*. Across three distinct synthesised structures the spread is 222 to 238 LUT4 (7%) and 41.76 to 44.96 MHz (7.6%), with dynamic power varying by less than 0.2%. Sklansky is the fastest by 4.7% but pays 16 additional LUT4 for it; Brent-Kung offers no advantage on this fabric, being marginally both larger and slower than Kogge-Stone. Flip-flop count is identical throughout.

This has a direct bearing on the architecture of [1]. At the widths characteristic of a biomedical datapath, the depth advantage of a minimum-depth prefix network amounts to roughly one logic level, and its effect is small compared with

TABLE V
PREFIX-ADDER COMPARISON (FOLDED 8-TAP FIR, IDENTICAL
OTHERWISE)

Prefix adder	LUT4	FF	$F_{\max}$ (MHz)	Power (mW)	Func.
Kogge-Stone [†]	222	44	42.956	2.595	PASS
Brent-Kung	223	44	41.762	2.595	PASS
Sklansky	238	44	44.955	2.599	PASS

[†]The Han-Carlson-scheduled description synthesises to this same netlist and is therefore not reported as a separate point (Section III-C).

the structural levers examined above: folding alone returns 35.7% of the LUT4 budget, and pipelining alone returns $1.55\times$ in frequency, whereas the entire span of the adder sweep is 7%. Effort spent selecting among prefix topologies at this width is, on this evidence, effort misdirected.

D. Comparison with the base architecture

Table VI places this work alongside the figures reported in [1]. **These rows are not like-for-like, and we do not present them as such.** Four differences must be held in mind.

- 1) **Different units of measurement.** Reference [1] characterises a *standalone unsigned multiplier*; the figures reported here characterise a *complete signed 8-tap FIR filter*, which contains four multipliers together with a delay line, pre-adders and a summation tree. A LUT count for one is not commensurable with a LUT count for the other.
- 2) **Different fabrics.** The base design targets a Xilinx Artix-7 (Nexys A7), whose 6-input LUTs differ structurally from the LUT4 primitives of the Gowin GW1NR-9C. Raw LUT counts do not transfer between the two.
- 3) **Different power methodology.** The 1.136 W reported in [1] and the 2.595 mW reported here differ by three orders of magnitude, which is not plausibly a design difference. The two figures are evidently obtained under different power-estimation assumptions, and *we therefore make no power claim of any kind against [1]*.
- 4) **Different frequency semantics.** The 0.714 MHz of [1] is consistent with the result rate of an iterative, multi-cycle multiplier, whereas the 42.96/66.47 MHz reported here is the maximum clock frequency of a parallel datapath. The gap reflects an architectural difference — iterative versus parallel evaluation — and not superior timing closure of the same structure.

E. The multiplier in isolation: an explicit area–time trade

The filter-level figures above do not answer the question a reader will ask directly: how large is the proposed multiplier on its own? We therefore synthesised the standalone 8×8 signed radix-4 MRPM through the same flow. It occupies **170 LUT4**, with zero registers, zero DSP blocks and 32 I/O ports, and is purely combinational.

We state the comparison plainly. The unsigned iterative multiplier of [1] occupies 68 LUT; the proposed signed parallel multiplier occupies 170 LUT4. *At the level of the multiplier*

alone, the proposed design is the larger of the two, and we make no claim to the contrary. Part of the difference is an artefact of the fabric — the 6-input LUTs of the Artix-7 absorb more logic per cell than the LUT4 primitives of the GW1NR-9C, so the raw ratio overstates the true gap — but the direction of the difference is not in doubt, and it is a direct and intended consequence of two design decisions.

The first is signedness. Two’s-complement operation requires sign extension through every partial product and a conditional negation (inversion and increment) at each Booth digit, none of which an unsigned datapath pays for. The second, and larger, is parallelism. The proposed multiplier instantiates four partial-product generators and a summation tree *in space*; the design of [1] instantiates a single adder and reuses it *in time* across approximately eight iterations. Spatial evaluation costs area by construction. This is the classical area–time trade-off, and the corresponding entry in the base design’s own results is its 0.714 MHz result rate: its compactness is purchased with sequential cycles. Comparing the area of an iterative multiplier with that of a parallel one, without quoting the throughput each achieves, is not a meaningful comparison.

Why this does not undermine the area claim. The standalone figure is, in any case, not the figure on which this paper’s area efficiency rests. Two observations follow. First, four standalone multipliers would account for roughly $4 \times 170 = 680$ LUT4, yet the complete folded filter — multipliers, delay line, pre-adders and summation tree together — occupies 222 LUT4. The difference is constant-coefficient specialisation: within the filter each multiplier’s second operand is a fixed coefficient ((1)), and the synthesiser collapses the corresponding logic aggressively, to the point of eliminating the $h_0 = 0$ multiplier altogether. A general-purpose multiplier and a coefficient-specialised one are therefore different objects, and the standalone number is a poor predictor of the deployed one.

Second, and decisively, the claim of area efficiency advanced in this paper is made against a like-for-like baseline rather than across fabrics: the same filter, on the same device, through the same toolchain, falls from 345 to 222 LUT4 — a reduction of 35.7% — with bit-identical output (Table IV). That measurement holds regardless of how the standalone multiplier compares with [1], and it is the measurement on which the title’s claim depends. What the present subsection establishes is the price of the architectural change: a larger multiplier, bought deliberately, in exchange for signed operation and a throughput roughly two orders of magnitude higher, and recovered at the filter level by folding.

The architectural comparison is summarised in Table VI.

F. Discussion

Three observations follow from the data. First, the structural levers dominate: folding returns 35.7% of the logic budget and pipelining returns $1.55\times$ in frequency, while the entire prefix-adder sweep spans 7%. An architecture that accelerates the adder inside an iterative loop, as in [1], is therefore optimising a term that is not the binding one; the partial-product count and the filter structure are.

TABLE VI
ARCHITECTURAL COMPARISON WITH THE BASE DESIGN [1]

	Base [1]	Proposed work
Standalone mult. area	68 LUT (6-LUT)	170 LUT4
Complete FIR area	not reported	222 LUT4
Multiplier rate	0.714 MHz (iter.)	single-cycle
Filter $F_{\max}$	not reported	42.96 / 66.47 MHz
Operands	unsigned	signed
Multiplier evaluation	~ 8 iterations	4 parallel PPs
Multipliers in FIR	8	4 (folded)
Pipelining	—	4-stage
Adder study	single (KS)	3 topologies swept
Target fabric	Artix-7 (Nexys A7)	Gowin GW1NR-9C
DSP blocks	—	0 (enforced)
Exactness	exact	exact (exhaustive)

Second, the folded design is strictly better than the direct design on every axis measured — 35.7% fewer LUT4, identical flip-flops, marginally higher $F_{\max}$, and bit-identical output — so there is no trade-off to negotiate. Folding should be regarded as the default for any linear-phase FIR.

Third, the pipelined folded design is smaller than the *direct* design (324 versus 345 LUT4) while running $1.55\times$ faster than the folded design, at the price of three additional cycles of latency. For a front-end ECG filter sampling at 250 Hz, a latency of four clock cycles at tens of megahertz is entirely negligible, so the pipelined configuration is the appropriate operating point for deployment.

VII. CONCLUSION

This paper has presented an area-efficient, exact, signed multiplier architecture for biomedical FIR filtering, extending the Modified Russian Peasant Multiplier of [1]. Radix-4 modified-Booth recoding replaces approximately eight sequential iterations with four partial products evaluated in parallel, and adds native two's-complement support. Exploiting the linear-phase symmetry of the coefficient set folds the 8-tap filter onto four multipliers rather than eight, reducing LUT4 utilisation by 35.7% (345 to 222) with provably bit-identical output. A four-stage pipeline raises the maximum clock frequency by $1.55\times$ (42.96 MHz to 66.47 MHz) for 115 additional flip-flops, leaving the pipelined folded filter both smaller and substantially faster than the unfolded, unpipelined design.

A comparative sweep of three synthesised prefix-adder topologies returned a largely negative but instructive result: at 8-bit datapath width the choice of prefix network spans only 7% in area and 7.6% in frequency, an order of magnitude less than the structural levers. This suggests that, for datapaths of the width typical in biomedical signal processing, effort is better spent on the partial-product count and the filter topology than on the adder — and, by implication, that the adder-centric optimisation pursued in [1] addresses a term that is not the binding constraint.

We also report, without qualification, the price of the architecture. In isolation the proposed signed parallel multiplier occupies 170 LUT4 against the 68 LUT of the unsigned

iterative design of [1]: at the level of the multiplier alone it is the larger of the two. That cost buys signed operation and single-cycle evaluation in place of an eight-iteration sequential loop, and it is recovered at the filter level, where coefficient specialisation and symmetric folding bring the complete filter to 222 LUT4.

Correctness was established without residual error tolerance: the multiplier is verified exhaustively over all 65,536 signed operand pairs with zero mismatches, and every filter configuration — folded, pipelined, and with each prefix adder — matches an independent golden model exactly. All results were obtained with DSP inference disabled and zero DSP blocks confirmed, so the reported area reflects genuine logic-fabric cost.

Future work. Two extensions follow directly. First, the results reported here are obtained at the place-and-route level; downloading the design to the device and exercising it on-chip would confirm the timing closure empirically. Second, the filter should be integrated into a complete acquisition chain and evaluated on real electrocardiogram and electroencephalogram recordings, so that denoising performance can be quantified on clinical signal-quality metrics rather than on synthetic vectors alone. A third direction, suggested by Section VI-E, is to quantify the area–time curve directly, by synthesising radix-2 and radix-8 variants of the multiplier alongside the radix-4 design reported here.

REFERENCES

- [1] TN, Deepika Sree, and S. Aditya. "A Low cost area-efficient modified Russian peasant multiplier (MRPM) for biomedical applications." *Integration* (2025): 102474.
- [2] Sharma, Mahendra, and Dr SK Singh. "New technique based Peasant multiplication for efficient signal processing applications." *Indonesian Journal of Electrical Engineering and Computer Science* 8, no. 3 (2017): 726-729.
- [3] Lotfabadi, Shahin Sanayei, Andy Ye, and Sridhar Krishnan. "Measuring the power efficiency of subthreshold FPGAs for implementing portable biomedical applications." *Microprocessors and Microsystems* 36, no. 3 (2012): 151-158.
- [4] Ajay, Arathi, and R. Mary Lourde. "VLSI Implementation of an improved multiplier for FFT Computation in Biomedical Applications." In *2015 IEEE Computer Society Annual Symposium on VLSI*, pp. 68-73. IEEE, 2015.
- [5] Phuse, Ashish S., and Pankaj P. Tasgaonkar. "Design and implementation of different multiplier techniques and efficient MAC unit on FPGA." In *2022 International Conference on Signal and Information Processing (IconSIP)*, pp. 1-5. IEEE, 2022.
- [6] Prabhu, A. S., and V. Elakya. "Design of modified low power booth multiplier." In *2012 International conference on computing, communication and applications*, pp. 1-6. IEEE, 2012.
- [7] Gokhale, G. R., and P. D. Bahirgonde. "Design of Vedic-multiplier using area-efficient Carry Select Adder." In *2015 International conference on Advances in computing, Communications and Informatics (ICACCI)*, pp. 576-581. IEEE, 2015.
- [8] Bansal, Yogita, Charu Madhu, and Pardeep Kaur. "High speed vedic multiplier designs-A review." *2014 Recent Advances in Engineering and Computational Sciences (RAECS)* (2014): 1-6.
- [9] Khurshid, Burhan. "FPGA-Based Resource-Optimal Approximate Multiplier for Error-Resilient Applications." *International Journal of Circuit Theory and Applications* 53, no. 9 (2025): 5518-5537.
- [10] Arunkumar, K., S. Sushma, and Vangala Siva Teja. "Improving Error Resilience in FPGA-Based Multi-Level Approximate Multipliers Using Modified Full Adders." In *2025 3rd International Conference on Device Intelligence, Computing and Communication Technologies (DICCT)*, pp. 153-158. IEEE, 2025.

- [11] Abed, Sa'ed, Bassam Jamil Mohd, Zaid Al-bayati, and Sahel Alouneh. "Low power Wallace multiplier design based on wide counters." *International Journal of Circuit Theory and Applications* 40, no. 11 (2012): 1175-1185.
- [12] Badawi, Aiman, Ali Alqarni, Abdullah Aljuffri, Mohammed S. Ben-Saleh, Abdulfattah M. Obeid, and Syed Manzoor Qasim. "FPGA realization and performance evaluation of fixed-width modified Baugh-Wooley multiplier." In *2015 Third International Conference on Technological Advances in Electrical, Electronics and Computer Engineering (TAECE)*, pp. 155-158. IEEE, 2015.
- [13] Paz, Pedro, and Mario Garrido. "Efficient implementation of complex multipliers on FPGAs using DSP slices." *Journal of Signal Processing Systems* 95, no. 4 (2023): 543-550.
- [14] Kumar, C. Uthaya, and B. Justus Rabi. "Design and implementation of modified russian peasant multiplier using msqrtsla based fir filter." *Indian Journal of Science and Technology* 9, no. 7 (2016): 1-6.
- [15] Gunasekaran, K., and M. Manikandan. "High speed reconfigurable FIR filter using Russian peasant multiplier with Sklansky adder." *Research Journal of Applied Sciences, Engineering and Technology* 8, no. 24 (2014): 2451-2456.
- [16] Rao, E. Jagadeeswara, T. Ramanjaneyulu, and K. Jayaram Kumar. "Advanced multiplier design and implementation using Han-Carlson adder." In *2018 International Conference on Intelligent and Innovative Computing Applications (ICONIC)*, pp. 1-5. IEEE, 2018.
- [17] Penchalaiah, U., and Siva Kumar Vg. "Design of high-speed and energy-efficient parallel prefix kogge stone adder." In *2018 IEEE International Conference on System, Computation, Automation and Networking (IC-SCA)*, pp. 1-7. IEEE, 2018.
- [18] Raju, Aradhanan, and Sudhir Kumar Sa. "Design and performance analysis of multipliers using Kogge Stone Adder." In *2017 3rd International Conference on Applied and Theoretical Computing and Communication Technology (iCATccT)*, pp. 94-99. IEEE, 2017.
- [19] Anjana, R., B. Abishna, M. S. Harshitha, E. Abhishek, V. Ravichandra, and M. S. Suma. "Implementation of vedic multiplier using Kogge-stone adder." In *2014 International Conference on Embedded Systems (ICES)*, pp. 28-31. IEEE, 2014.
- [20] Sipeed, "Tang Nano 9K FPGA development board (Gowin GW1NR-9) datasheet," 2023. [Online]. Available: <https://wiki.sipeed.com/>